The Continuum Shadow (CODA-CS)

Subtitle: Continuum Re-Presentations that Descend to the Finite Calculus

Author: Jeremy H. Carroll

Date: 2026-09-13

Version: v1.1 Pre-Print

License: Creative Commons Attribution 4.0 International

CODA Architecture & Layers

CODA stands for the Calculus of Obstruction, Descent, and Authorization. It is an exact, finite mathematical apparatus that enforces strict claim discipline by separating linear repairs, transport certificates, and finite descent from ungrounded authority claims. The system comprises eight operational layers governed by a single overarching authority architecture (GTLA).

Layer	Abbreviation	Layer / Component Name & Scope
1	**FOC**	Finite Obstruction Calculus
2	**MUR**	Meta & Universal Re-Presentation (what presentations exist)
3	**CS**	Continuum Shadow (what continuum presentation may descend to a finite calculus system)
4	**CS_Q**	Continuum Shadow to Continuum Bridge (how continuum presentations descend)
5	**Q**	The Continuum (continuum presentations via completion and enclosure)
6	**A**	Anomalon (global residuals not removed and typed instead)
7	**OR**	Omniversal Re-Presentation (what presentations can transport)
8	**FDA**	Finite Descent Authority (what presentation may descend to evaluation and authorization)
9	**GTLA**	Layered Authority Architecture for CODA

Abstract

A fine observation can be forgotten to a coarse one: compose a forgetful map after the fine look. If a query is already constant on the coarse fibres, it is constant on the fine fibres. Two family observations agree if and only if they agree at every index. Four Boolean pairs share two first coordinates; the first coordinate does not determine the second. Finite observations may fail to determine a query, and a missing terminalizer blocks query authority. Writing a continuum name, or writing a limit, does not by itself confer authority. Authorized glue of overlapping shadows is not proved here.

Introduction

It is easy to treat a blurry photo as the original. You sample a signal on a grid; you keep the first coordinate of a pair; you write $\lim_{n\to\infty}$ and talk as if that notation had answered the question you actually asked. The coarse picture may be honest, and still forget something.

A picture. States are Boolean pairs $X = \mathbf{2} \times \mathbf{2}$. The fine look is the identity: you see both coordinates. The coarse look is the first coordinate. Four dots collapse onto two. The query "what is the second coordinate?" is constant on the identity fibres — each fibre is a singleton — and is not constant on the first-coordinate fibres: $(\mathsf{false}, \mathsf{false})$ and $(\mathsf{false}, \mathsf{true})$ share a first coordinate and disagree on the second.

A second picture. You have a fine look $\mathrm{fine} : X \to B$ and a forgetful map $\mathrm{forget} : B \to A$. The composite $\mathrm{forget} \circ \mathrm{fine}$ is the coarse look. If a query $q : X \to Y$ cannot tell apart states that already look the same after forgetting, then it also cannot tell apart states that look the same before forgetting. Refinement preserves descent.

The theory is refuted by any of: a query constant on coarse fibres that splits a fine fibre; two family observations that agree as functions but disagree at some index; a factor of the first-coordinate look that recovers the second coordinate; a query-authority witness extracted from limit notation alone.

Notation

Throughout, the types are ordinary (they need not be finite).

- X is a type of states.
- A and B are types of observations. B is finer; A is coarser.
- $\mathrm{fine} : X \to B$ is a fine look. $\mathrm{forget} : B \to A$ forgets.
- The coarse look is the composite $\mathrm{forget} \circ \mathrm{fine} : X \to A$.
- $q : X \to Y$ is a query: a function of the state whose value you want.
- q is *fibre-constant* for a look $p : X \to P$ when $p(x) = p(y)$ implies $q(x) = q(y)$.
- q *factors through* p when there is $g : P \to Y$ with $g(p(x)) = q(x)$ for every x.
- A family of observations indexed by I is a map $(i : I) \to (X \to P_i)$.

A **refinement** of a coarse look $p : X \to A$ is a pair $(\mathrm{fine}, \mathrm{forget})$ with $p = \mathrm{forget} \circ \mathrm{fine}$.

1. Forgetting a fine look

Definition CS-D1 — fibre constancy

$$\mathsf{FiberConstant}(p, q) : \Longleftrightarrow\ \forall x, y.\ p(x) = p(y)\ \Rightarrow\ q(x) = q(y).$$

The Lean predicate is `Coda.FiberConstant`.

Definition CS-D2 — factorization

$$\mathsf{Factors}(p, q) : \Longleftrightarrow\ \exists g : P \to Y.\ \forall x.\ g(p(x)) = q(x).$$

The Lean predicate is `Coda.Factors`. Factorization is a function of the observation. Fibre constancy is the consequence that equal observations give equal answers.

Definition CS-D3 — refinement

A refinement of $p : X \to A$ is a fine look $\mathrm{fine} : X \to B$ and a forgetful map $\mathrm{forget} : B \to A$ such that

$$p = \text{forget} \circ \text{fine}.$$

In pictures: take the fine photo, then blur it. The composite is the coarse photo.

$$\begin{array}{ccc} X & \xrightarrow{\text{fine}} & B \\ & \searrow_{\text{forget}\circ\text{fine}} & \downarrow \text{forget} \\ & & A \end{array}$$

No extra data is created by writing the composite.

Lemma CS-L1 — factorization implies fibre constancy

If $\mathsf{Factors}(p, q)$, then $\mathsf{FiberConstant}(p, q)$.

Proof. Take a factor g. If $p(x) = p(y)$, then $q(x) = g(p(x)) = g(p(y)) = q(y)$. This is Lean `Coda.factor_implies_constant`. □

The converse, through P itself, can fail if p is not surjective: a function defined on the image $p(X) \subseteq P$ need not extend to a total map on P without additional choices. (When the target Y is nonempty and a classical default value can be chosen outside the image, a total extension exists, but the distinction between image factorization and a canonical total decoder remains relevant.) Through the image of p, fibre constancy is sufficient to define a factor; that image lemma is not needed below.

2. If it already agrees coarsely, it agrees finely

Theorem CS-T1 — refinement preserves descent

If q is fibre-constant for forget ∘ fine, then q is fibre-constant for fine.

Proof. Let $\text{fine}(x) = \text{fine}(y)$. Apply forget:

$$\text{forget}(\text{fine}(x)) = \text{forget}(\text{fine}(y)).$$

The hypothesis says $q(x) = q(y)$. This is Lean `Coda.CS.refinement_preserves_descent`. □

In words: if the query already cannot see a distinction after blurring, it cannot see that distinction in the sharp photo either. The sharp photo has smaller fibres. Constancy on the larger fibres is the stronger hypothesis.

A positive instance. Let $X = \mathbf{2} \times \mathbf{2}$, let fine be the identity, let forget be first-coordinate projection, and let q itself be first-coordinate projection. Then q equals the coarse look, so it is fibre-constant there, and Theorem CS-T1 gives fibre constancy on the identity.

A non-instance, not a counterexample. The second-coordinate query is *not* fibre-constant on the coarse look, so the hypothesis of Theorem CS-T1 fails. Section 4 records that failure as a collision.

3. Family observations agree pointwise

A family of looks, one per index $i \in I$, is a function of two arguments: the index and the state. Two states can be compared by comparing the whole family at once, or by comparing each look separately.

Theorem CS-T2 — family observation equality is pointwise

Let observe : $(i : I) \to (X \to P_i)$ and let $x, y \in X$. Then

$$(i \mapsto \text{observe}_i(x)) = (i \mapsto \text{observe}_i(y)) \quad \text{if and only if} \quad \forall i.\ \text{observe}_i(x) = \text{observe}_i(y).$$

Proof. Left to right: apply the two functions at i. Right to left: pointwise equality of functions is function extensionality. This is Lean `Coda.CS.family_observation`. □

Sampling a signal at a list of times is a family of observations. Two signals give the same sample list if and only if they agree at each sample time. There is no extra agreement, sitting in the list as a whole, that the pointwise comparisons missed. Writing the list as a single vector does not create a new observation.

4. Four dots that look the same

Let $X = \mathbf{2} \times \mathbf{2}$. The four states, with the two projections, are:

pair	first	second
(false, false)	false	false
(false, true)	false	true
(true, false)	true	false
(true, true)	true	true

The first-coordinate look has two fibres, each of size two:

$$\text{forget}^{-1}(\mathsf{false}) = \{(\mathsf{false}, \mathsf{false}), (\mathsf{false}, \mathsf{true})\},$$

$$\text{forget}^{-1}(\mathsf{true}) = \{(\mathsf{true}, \mathsf{false}), (\mathsf{true}, \mathsf{true})\}.$$

The identity look has four fibres, each a singleton. On the false fibre the second coordinate takes both values. That is the collision.

Theorem CS-T3 — the first coordinate does not determine the second

The second-coordinate query does not factor through first-coordinate projection:

$$\neg\mathsf{Factors}(x \mapsto x.1,\ x \mapsto x.2).$$

Proof. The pair (false, false) and the pair (false, true) have the same first coordinate and different second coordinates. Lemma CS-L1 would make the seconds agree. This is Lean `Coda.CS.finite_shadow_collision`, by `Coda.collision_obstructs_factor`. □

Four dots, two shadows. The shadow does not recover the forgotten bit.

Theorem CS-T4 — the fine look can factor when the coarse look cannot

The second-coordinate query factors through the identity look, and does not factor through first-coordinate projection:

$$\mathsf{Factors}(x \mapsto x,\ x \mapsto x.2) \quad \text{and} \quad \neg\mathsf{Factors}(x \mapsto x.1,\ x \mapsto x.2).$$

Proof. The factor through the identity is second-coordinate projection itself. The negative conjunct is Theorem CS-T3. This is Lean `Coda.CS.refinement_strict_countermodel`. □

Refinement can split a fibre that the coarse look collapsed. That is the whole gap between the blurry photo and the original. It is not a licence to treat the original as having been recovered from the blur.

5. Limit notation is not an answer

A named continuum object — a signal X, a sequence, a written limit $\lim_{n\to\infty} x_n$ — is a name. A query q is a function you would like to evaluate. Finite observations may fail to determine a query (Theorems CS-T3 and CS-T4); a missing terminalizer blocks query authority within the calculus (Theorem CS-T5).

Theorem CS-T5 — missing terminalizer blocks query authority

For every signal X, query q, and bridge record b, if $b.\text{has_local_terminalizer} = \mathsf{false}$, then $\neg\mathsf{HasQueryAuthority}(X, q, b)$, where $\mathsf{HasQueryAuthority}$ is the conjunction of the eight declared admissibility flags. In particular, writing a continuum object and a query does not manufacture authority when the terminalizer flag is missing.

Proof. The authority predicate $\mathsf{HasQueryAuthority}(X, q, b)$ evaluates $b.\text{isAllAdmissible} = \mathsf{true}$, requiring all eight flags to hold simultaneously. When $b.\text{has_local_terminalizer} = \mathsf{false}$, the conjunction evaluates to false. This is the pointwise Lean theorem `terminalizer_absence_blocks_query_authority` in `CodaCs`. The existential form `continuum_limit_does_not_create_authority` witnesses this with `unadmitted_bridge_packet`. □

The witness is small on purpose. The constant-zero query has a uniquely determined mathematical value that factors through every profile, yet query authority is refused because the authorization flag is missing: lack of authority is an administrative refusal, not mathematical nondetermination of values (which is governed by the fibre collision in Theorem CS-T3). Limit notation is the same kind of naming. It does not fill the missing flag, and it does not pick the forgotten second coordinate of Section 4. A family of ever-finer sample grids is a family of observations in the sense of Theorem CS-T2: agreement is still pointwise at each grid. The written limit is not an extra sample.

6. Computational companion

The production module `code/coda_cs_core.py` records finite sample profiles of named signals, checks fibre constancy on a finite list of test signals, and refuses a query when a declared local terminalizer is missing. Arithmetic is exact rational; floats and booleans in scalar position raise `TypeError`.

The shipped quadrature on the grid $\{0, 1/4, 1/2, 3/4, 1\}$ with $X(t) = 2t$ at those nodes is

$$0^2 \cdot \frac{1}{4} + \left(\tfrac{1}{2}\right)^2 \cdot \frac{1}{4} + 1^2 \cdot \frac{1}{4} + \left(\tfrac{3}{2}\right)^2 \cdot \frac{1}{4} = \frac{7}{8}.$$

Two signals that match on the grid match on that query. Dropping the nodes $1/4$ and $3/4$ is forget after the five-point look: a second signal may then differ at the dropped nodes and still match the coarse

energy, while a query that reads $X(1/4)$ does not. That is a finite replay of fibre constancy and of Theorem CS-T3, not a theorem about every function $[0,1] \to \mathbb{R}$.

Lean files in `lean4/`:

- records for signals, profiles, and shadow packets;
- fibre constancy from factorization;
- `coda_cs_algebra.lean` — self-contained definitions and proofs of Definitions CS-D1–D3, Lemma CS-L1, and Theorems CS-T1–T4;
- `coda_cs_l01_continuum_non_creation.lean` — Theorem CS-T5;
- `coda_cs_000.lean` — re-exports the files above.

There are no `sorry` stubs in those files. The reconstruction theorems use the ordinary Lean kernel.

Adversarial tests include: two signals with the same profile and different query values; a factored evaluator that disagrees with the query; a missing local terminalizer; leftover 0 and a positive leftover on a two-row cochain example; float rejection.

Software package version `1.1.0` corresponds to Manuscript `v1.1` Pre-Print (previously indexed under internal engine tag `3.0.0`; synchronized to `1.1.0` across `pyproject.toml`, `CITATION.cff`, and Lake for release distribution consistency).

7. Limitations

CS-T1–CS-T4 concern refinement, pointwise observation equality, and explicit profile collisions. CS-T5 states that a missing terminalizer makes the declared eight-flag query-authority predicate false. It does not imply that the query lacks a uniquely determined mathematical value. They hold for ordinary functions of types.

Authorized glue of overlapping shadows — the H3'' lemma — is not proved here.

A written limit is a name. Theorem CS-T5 is a refusal to treat that name as a query-authority witness, not a construction of a topological limit, and not a uniqueness theorem for actual analysis.

References

1. S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998. https://doi.org/10.1007/978-1-4757-4721-8
2. A. Grothendieck, "Technique de descente et théorèmes d'existence en géométrie algébrique," Séminaire Bourbaki, 1959.
3. J. H. Carroll, *Meta & Universal Re-Presentation: Commuting Squares, Composition, and Irreversible Forgetting: A commuting square is not target standing (CODA-MUR)*, 2026.
4. J. H. Carroll, *Omniversal Re-Presentation: Re-Presentations Transport Rules (CODA-OR)*, 2026.
5. P. Cousot and R. Cousot, "Abstract interpretation: a unified lattice model for static analysis of programs," in *POPL*, 1977.